

A
Project Report
on
**CIFAKE Image Classification and Explainable Identification of
AI-Generated Synthetic Images**

Submitted to
XtraGrad Pvt. Ltd

AICTE INTERNSHIP

By

**“Niraj Khankari
AICTE STU ID: STU6867ed37a45b01751641399 ”**

Under the guidance of
Abhienaya Sri Soundarajan

Project Github Link :
**[https://github.com/NirajKhankari30/CIFAKE-Image-Classification-and-Explainable-Id
entification-of-AI-Generated-Synthetic-Images](https://github.com/NirajKhankari30/CIFAKE-Image-Classification-and-Explainable-Identification-of-AI-Generated-Synthetic-Images)**

INDEX

CHAPTER 1: INTRODUCTION

CHAPTER 2: SYSTEM OVERVIEW

CHAPTER 3: LITERATURE SURVEY

CHAPTER 4: TECHNOLOGIES USED

CHAPTER 5: ARCHITECTURE AND DESIGN

CHAPTER 6: IMPLEMENTATION

CHAPTER 7: RESULTS AND DISCUSSION

CHAPTER 8: CHALLENGES AND LEARNING OUTCOMES

CHAPTER 9: FUTURE ENHANCEMENTS

CHAPTER 10: CONCLUSION

CHAPTER 1: INTRODUCTION

1.1 Background

Artificial Intelligence (AI) has made significant progress in the field of image generation. Modern AI-based image generation techniques can create images that look very similar to real photographs. These synthetic images are useful in areas such as digital content creation, entertainment, advertising, education, and design. However, the increasing quality of AI-generated images has also created challenges in distinguishing them from genuine images.

The ability to identify whether an image is real or artificially generated has therefore become an important research problem. Human observation alone may not always be sufficient because synthetic images can contain details that appear realistic to the human eye. Automated image classification techniques can help identify patterns that may not be easily noticeable.

This project, titled “CIFAKE Image Classification and Explainable Identification of AI-Generated Synthetic Images,” focuses on developing a deep learning-based system that classifies images into two categories: FAKE and REAL. The project uses the CIFAKE dataset and a Convolutional Neural Network (CNN) for image classification.

In addition to classification, the project also incorporates Explainable AI (XAI) using Grad-CAM (Gradient-weighted Class Activation Mapping). Grad-CAM helps visualize the regions of an image that contributed to the model's prediction. This makes the system more interpretable and provides a visual explanation rather than only displaying the predicted class.

1.2 Problem Statement

With the rapid development of AI image-generation technologies, distinguishing AI-generated images from real images has become increasingly difficult. Traditional manual inspection may not provide reliable results, especially when synthetic images are highly realistic.

The problem addressed in this project is to develop an automated system capable of:

- Classifying images as FAKE or REAL.
- Learning visual patterns from a labelled image dataset.
- Evaluating the classification performance using standard metrics.
- Providing an understandable visual explanation for individual predictions.

1.3 Objectives

The main objectives of the project are:

1. To understand the characteristics of real and AI-generated images.
2. To prepare and validate the CIFAKE dataset for deep learning.
3. To preprocess and resize images into a consistent format.
4. To develop a CNN-based binary image classification model.

5. To train and validate the model using appropriate techniques.
6. To evaluate the trained model using accuracy, precision, recall, F1-score and confusion matrix.
7. To identify correctly classified and misclassified images.
8. To analyze prediction confidence and uncertain predictions.
9. To implement Grad-CAM for visual explanation of model predictions.
10. To save and organize the trained model, evaluation results, graphs and xAI outputs.

1.4 Significance of the Project

The project demonstrates how deep learning and explainable AI can be combined to address a practical image-analysis problem. Classification provides an automated prediction, while Grad-CAM provides additional insight into the decision-making process of the CNN.

This combination is useful because a classification result without explanation can be difficult to interpret. Visual explanations can help students, researchers and developers better understand what the model has learned from the input images.

CHAPTER 2: SYSTEM OVERVIEW

2.1 Introduction

The developed system is an image classification and explainability pipeline designed to distinguish between real and AI-generated images. The system takes an image dataset as input, processes the images, trains a CNN model, evaluates its performance, and generates visual explanations using Grad-CAM.

The complete workflow consists of several stages. First, the dataset is examined for class distribution, image dimensions, colour modes and corrupted files. The images are then converted into a consistent format and resized to 32×32 pixels.

After preprocessing, the data is divided into training, validation and testing sets. A CNN model is trained using the training set, while the validation set is used to monitor the learning process.

Once training is complete, the best model is evaluated on unseen test images. The predictions are compared with the actual labels to calculate various performance metrics.

Finally, Grad-CAM is applied to selected images to visualize the image regions that contributed to the model's prediction.

2.2 System Workflow

The overall workflow can be summarized as:

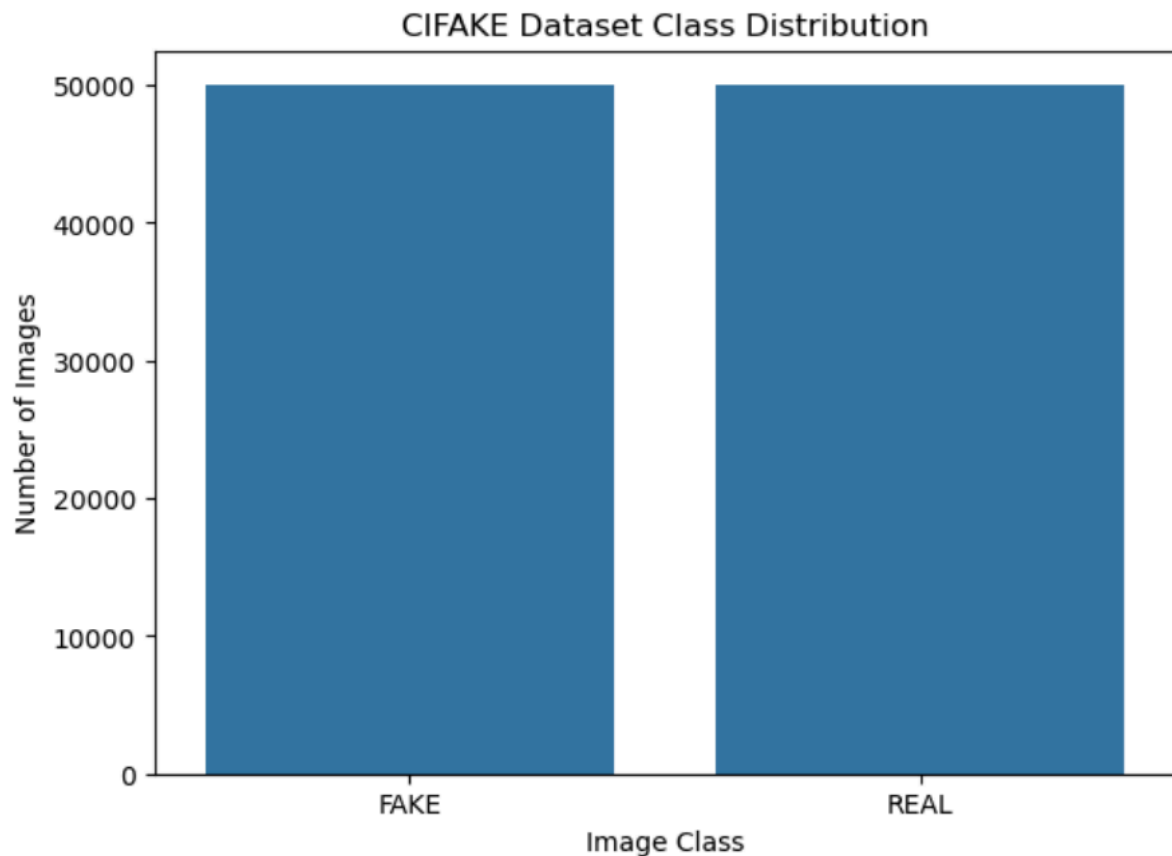
Dataset → Data Validation → Preprocessing → Data Augmentation → Train/Validation/Test Split → CNN Training → Model Selection → Testing → Performance Evaluation → Grad-CAM Explanation → Output Saving

2.3 Input

The input to the system is an image belonging to either of the two categories:

- FAKE – AI-generated or synthetic images.
- REAL – genuine images.

The original dataset contains separate folders for these two classes.



2.4 Processing

The major processing operations include:

- Image file validation.
- Detection of corrupted images.

```
corrupted_fake = check_corrupted_images(fake_path)
corrupted_real = check_corrupted_images(real_path)

print("Corrupted FAKE images:", len(corrupted_fake))
print("Corrupted REAL images:", len(corrupted_real))
```

Corrupted FAKE images: 45000

Corrupted REAL images: 45000

- Image resizing.

```
Image Height : 32
Image Width  : 32
Channels      : 3
Batch Size   : 64
Classes      : 2
```

- RGB conversion.
- Pixel normalization.
- Dataset splitting.
- Training-time image augmentation.
- CNN feature extraction.
- Classification.

2.5 Output

The system generates:

- Predicted class: FAKE or REAL.
- Probability associated with the REAL class.
- Prediction confidence.
- Confusion matrix.
- Accuracy.
- Precision.
- Recall.
- F1-score.
- Error rate.
- Training and validation graphs.
- Misclassified image information.
- Grad-CAM visual explanations.
- Saved trained model and result files.

CHAPTER 3: LITERATURE SURVEY

3.1 Introduction

The detection of synthetic and AI-generated images has become an important area of research due to the rapid development of generative AI techniques. Researchers have explored different approaches for detecting manipulated or generated content, including traditional image-processing techniques, machine learning, deep learning and explainable AI.

3.2 Traditional Image-Based Detection

Early approaches to image authenticity detection often focused on manually designed features. These approaches examined properties such as image noise, compression patterns, texture, colour inconsistencies and other statistical characteristics.

Although these methods can identify certain forms of manipulation, their performance may decrease when dealing with modern AI-generated images because generative models can produce highly realistic visual patterns.

3.3 Machine Learning Approaches

Machine learning algorithms have also been used for image classification. Features can be extracted from images and supplied to classifiers such as Support Vector Machines, Random Forests and other conventional algorithms.

These methods can be effective when appropriate features are available. However, designing suitable features manually can be difficult for complex image-generation problems.

3.4 Deep Learning and CNNs

Deep learning has significantly improved image classification by allowing models to learn useful features directly from image data.

Convolutional Neural Networks are particularly suitable for image-related tasks because convolutional layers can learn spatial patterns such as edges, textures, shapes and increasingly complex visual structures.

In the present project, a CNN is used to learn visual patterns from the CIFAKE dataset and classify images into FAKE and REAL categories.

3.5 Explainable Artificial Intelligence

Although deep learning models can achieve strong classification performance, they are often considered difficult to interpret because their internal decision-making process is not directly visible.

Explainable AI attempts to make model predictions easier to understand. Techniques such as Grad-CAM can highlight image regions that have a strong influence on a CNN's prediction.

In this project, Grad-CAM is used to complement the CNN classification result by providing a visual representation of the areas considered important by the model.

3.6 Research Gap

A classification model can provide a prediction, but simply reporting the prediction does not explain why the model made that decision. Therefore, combining image classification with an explainability technique provides a more informative system.

This project addresses this requirement by combining a CNN-based classifier with Grad-CAM-based visual interpretation.

CHAPTER 4: TECHNOLOGIES USED

4.1 Python

Python was used as the primary programming language for the project. Its extensive collection of libraries makes it suitable for data processing, machine learning, deep learning and visualization.

4.2 Jupyter Notebook

Jupyter Notebook was used as the main development environment. It allowed the project to be developed step by step, with code, output, graphs and observations maintained together.

This was particularly useful during dataset analysis, model training and debugging.

4.3 TensorFlow and Keras

TensorFlow with Keras was used to develop and train the CNN model.

Keras provides high-level APIs for defining neural network architectures, compiling models, training them and saving trained models.

4.4 NumPy

NumPy was used for numerical operations and manipulation of arrays. It was also used while processing predictions, labels and model outputs.

4.5 Pandas

Pandas was used for organizing dataset information and storing prediction and evaluation results in tabular form.

For example, image paths, class labels, prediction probabilities and predicted classes were stored using Pandas DataFrames.

4.6 Matplotlib

Matplotlib was used to create visualizations such as:

- Class distribution plots.
- Training accuracy curves.
- Validation accuracy curves.
- Training loss curves.
- Validation loss curves.
- Image visualization.

- Grad-CAM outputs.

4.7 Seaborn

Seaborn was used for visualizing the confusion matrix in the form of a heatmap.

4.8 Scikit-learn

Scikit-learn was used for model evaluation. The project used functions for calculating:

- Accuracy.
- Precision.
- Recall.
- F1-score.
- Confusion matrix.
- Classification report.

4.9 PIL

The Python Imaging Library (PIL) was used for image handling and validation. It was also used to check image properties and detect corrupted image files.

4.10 Development Environment

The project was developed using:

- Python
- Anaconda
- Jupyter Notebook
- TensorFlow/Keras
- Windows operating system

CHAPTER 5: ARCHITECTURE AND DESIGN

5.1 System Architecture

The proposed system follows a sequential machine learning pipeline. Each stage prepares the output required by the next stage.

The architecture consists of the following major components:

1. Dataset Layer
2. Data Validation Layer
3. Preprocessing Layer
4. Data Augmentation Layer
5. CNN Model Layer
6. Prediction Layer
7. Evaluation Layer
8. Explainability Layer
9. Output and Storage Layer

5.2 Dataset Layer

The CIFAKE dataset contains two categories of images:

- FAKE
- REAL

The classes were mapped numerically as:

Class	Label
-------	-------

FAKE	0
------	---

REAL	1
------	---

5.3 Data Preprocessing Layer

Images are processed before being provided to the CNN.

The preprocessing steps include:

- Reading the image.
- Decoding the image.
- Converting it to RGB.
- Converting pixel values into floating-point representation.

- Resizing images to 32×32 pixels.

5.4 Data Augmentation Layer

Training-time augmentation was used to introduce small variations in the training images.

The augmentation operations include:

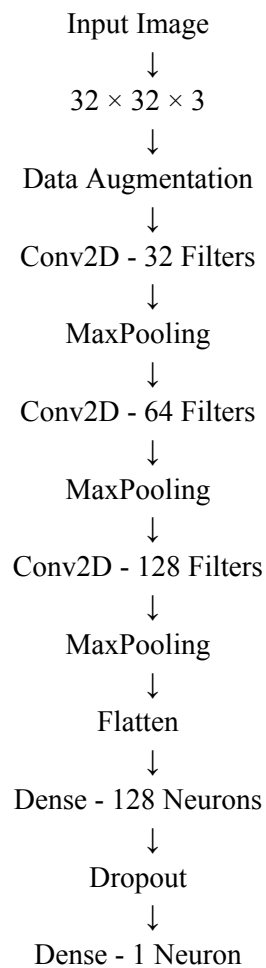
- Random horizontal flipping.
- Random rotation.
- Random zoom.

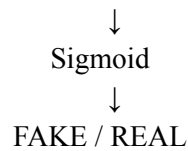
The augmentation layer was included inside the model so that validation and test images remained unaffected by augmentation.

5.5 CNN Architecture

The implemented CNN contains three convolutional feature-extraction stages followed by fully connected layers.

The main architecture is:





5.6 Classification Layer

The final layer contains a single neuron with a sigmoid activation function.

The output represents the probability of the REAL class.

Therefore:

- Probability close to 0 $\rightarrow$ FAKE.
- Probability close to 1 $\rightarrow$ REAL.

A threshold of 0.5 was used for classification.

5.7 Explainability Layer

Grad-CAM was implemented using the final convolutional layer, conv2d_2.

The purpose of this layer in the explainability pipeline is to identify the spatial features that contributed to the classification result.

The resulting heatmap is combined with the original image to create an overlay that visually highlights important regions.

```
model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
data_augmentation (Sequential)	(None, 32, 32, 3)	0
conv2d (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_1 (Conv2D)	(None, 16, 16, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
conv2d_2 (Conv2D)	(None, 8, 8, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 128)	262,272
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 1)	129

Total params: 355,649 (1.36 MB)

Trainable params: 355,649 (1.36 MB)

Non-trainable params: 0 (0.00 B)

CHAPTER 6: IMPLEMENTATION

6.1 Dataset Preparation

The project began by loading the image dataset and checking the availability of the FAKE and REAL folders.

The number of images in each class was examined to understand the dataset distribution.

Images were also inspected manually to verify that the dataset contained appropriate examples.

6.2 Dataset Validation

Before training, image files were checked for possible corruption using image verification techniques.

The dimensions and colour modes of the images were also examined.

This step helped ensure that invalid files did not interfere with model training.

```
val_loss, val_accuracy = best_model.evaluate(
    val_dataset,
    verbose=1
)
```

```
print("\nValidation Loss    :", val_loss)
print("Validation Accuracy:", val_accuracy)
```

24/24 ————— 1s 27ms/step - accuracy: 0.8857 - loss: 0.2616

Validation Loss : 0.26496633887290955

Validation Accuracy: 0.8853333592414856

6.3 Dataset Splitting

The dataset was divided into three subsets:

- 70% Training
- 15% Validation
- 15% Testing

A stratified split was used so that both classes were represented appropriately in the different subsets.

6.4 Image Preprocessing

All input images were resized to:

$32 \times 32 \times 3$

where:

- 32 = image height.
- 32 = image width.
- 3 = RGB colour channels.

Pixel values were converted to floating-point values suitable for neural network processing.

6.5 Data Augmentation

The CNN uses random image augmentation during training.

The implemented transformations were:

Random Horizontal Flip

Random Rotation

Random Zoom

These transformations provide small variations in the training data and help the model learn more general visual patterns.

6.6 CNN Model Development

The CNN consists of three convolutional layers with increasing numbers of filters:

- 32 filters
- 64 filters
- 128 filters

Max-pooling layers were placed after each convolutional layer.

The extracted features were then flattened and passed through a dense layer containing 128 neurons.

A dropout layer with a rate of 0.5 was used before the final classification layer.

The final layer uses sigmoid activation for binary classification.

6.7 Model Compilation

The model was compiled using:

- Optimizer: Adam
- Learning Rate: 0.001
- Loss Function: Binary Cross-Entropy
- Evaluation Metric: Accuracy

Binary cross-entropy is suitable for the two-class classification problem used in this project.

6.8 Model Training

The model was trained for a maximum of 20 epochs.

Early stopping was used to prevent unnecessary training when validation performance stopped improving.

Model checkpointing was also implemented so that the model with the best validation accuracy could be saved.

The best validation accuracy was achieved at Epoch 4, with a validation accuracy of 88.53%.

Training the CNN

```
EPOCHS = 20

history = model.fit(
    train_dataset,
    validation_data=val_dataset,
    epochs=EPOCHS,
    callbacks=[
        early_stopping,
        model_checkpoint
    ]
)

Epoch 1/20
109/110 ————— 0s 45ms/step - accuracy: 0.6418 - loss: 0.6068
Epoch 1: val_accuracy improved from -inf to 0.76200, saving model to cifake_best_model.keras
110/110 ————— 9s 53ms/step - accuracy: 0.6435 - loss: 0.6051 - val_accuracy: 0.7620 - val_loss: 0.5184
Epoch 2/20
110/110 ————— 0s 46ms/step - accuracy: 0.8397 - loss: 0.3681
Epoch 2: val_accuracy did not improve from 0.76200
110/110 ————— 6s 52ms/step - accuracy: 0.8397 - loss: 0.3681 - val_accuracy: 0.7407 - val_loss: 0.6152
Epoch 3/20
110/110 ————— 0s 56ms/step - accuracy: 0.8657 - loss: 0.3066
Epoch 3: val_accuracy improved from 0.76200 to 0.79067, saving model to cifake_best_model.keras
110/110 ————— 7s 62ms/step - accuracy: 0.8657 - loss: 0.3066 - val_accuracy: 0.7907 - val_loss: 0.5332
Epoch 4/20
109/110 ————— 0s 57ms/step - accuracy: 0.8839 - loss: 0.2799
Epoch 4: val_accuracy improved from 0.79067 to 0.88533, saving model to cifake_best_model.keras
110/110 ————— 7s 63ms/step - accuracy: 0.8839 - loss: 0.2799 - val_accuracy: 0.8853 - val_loss: 0.2650
- - - - -
```

6.9 Model Prediction

For an input image, the trained model produces a sigmoid probability.

For example, if the output is:

0.03

the model estimates approximately 3% probability for REAL and therefore approximately 97% probability for FAKE.

A threshold of 0.5 is used to convert the probability into the final class prediction.

6.10 Model Evaluation

After training, predictions were generated for all 1,500 test images.

The predictions were compared with their actual labels.

The following evaluation measures were calculated:

- Accuracy
- Precision
- Recall
- F1-score
- Error rate

- Confusion matrix

6.11 Grad-CAM Implementation

Grad-CAM was implemented to provide a visual explanation for the CNN's prediction.

The final convolutional layer was used to obtain feature maps. Gradients of the predicted output with respect to these feature maps were calculated.

The gradients were then used to determine which spatial regions contributed more strongly to the prediction.

The resulting heatmap was resized and overlaid on the original image.

This produced an interpretable visualization of the model's attention.

CHAPTER 7: RESULTS AND DISCUSSION

7.1 Model Performance

The final CNN model was evaluated using 1,500 previously unseen test images.

The obtained results were:

Metric	Result
Test Images	1,500
Correct Predictions	1,351
Incorrect Predictions	149
Accuracy	90.07%
Precision	95.88%
Recall	83.73%
F1-Score	89.40%
Error Rate	9.93%
Best Epoch	4
Best Validation Accuracy	88.53%

These results show that the CNN was able to learn useful visual patterns for distinguishing the two classes in the CIFAKE test set.

7.2 Confusion Matrix

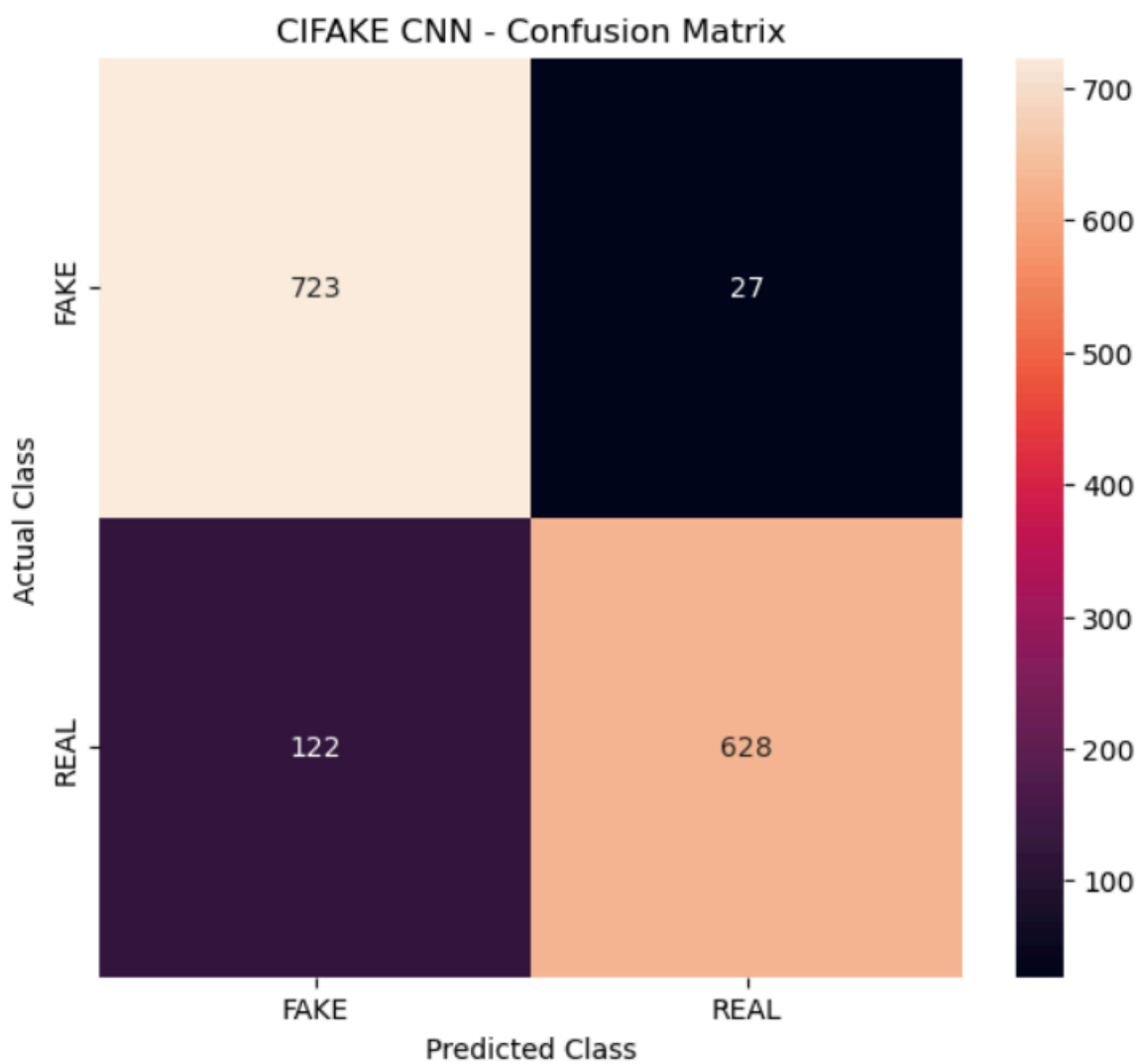
The confusion matrix obtained from the test dataset was:

```
[[723  27]
 [122 628]]
```

CONFUSION MATRIX

```
[[723  27]
 [122 628]]
```

True Negatives (TN): 723
False Positives (FP): 27
False Negatives (FN): 122
True Positives (TP): 628



The interpretation is:

- 723 FAKE images were correctly classified as FAKE.
- 27 FAKE images were classified as REAL.
- 122 REAL images were classified as FAKE.

- 628 REAL images were correctly classified as REAL.

The model therefore correctly classified 1,351 out of 1,500 images.

7.3 Precision

The precision obtained was 95.88%.

Precision indicates how many of the images predicted as REAL were actually REAL under the project's class definition.

The relatively high precision indicates that false REAL predictions were less frequent than the number of correct REAL predictions.

7.4 Recall

The recall obtained was 83.73%.

Recall measures how many of the actual REAL images were successfully identified as REAL.

The difference between precision and recall indicates that the model made more errors in which REAL images were classified as FAKE than the reverse error.

7.5 F1-Score

The F1-score was 89.40%.

The F1-score combines precision and recall into a single measure and is useful when both types of classification performance need to be considered.

7.6 Training Performance

The training and validation accuracy graphs were generated to observe how the model learned over the training epochs.

Similarly, training and validation loss graphs were generated to examine changes in the model's error during training.

Early stopping and model checkpointing were used to select a suitable trained model based on validation performance.

```

=====
FINAL CIFAKE MODEL RESULTS
=====
Total Test Images           : 1500
Correct Predictions         : 1351
Incorrect Predictions       : 149
Accuracy                    : 90.07%
Precision                   : 95.88%
Recall                      : 83.73%
F1-Score                    : 89.40%
Error Rate                  : 9.93%
Best Epoch                  : 4
Best Validation Accuracy    : 88.53%

✓ Training accuracy saved
✓ Training loss saved
✓ Confusion matrix image saved
✓ Confusion matrix CSV saved
✓ Classification report saved
✓ Test predictions CSV saved
✓ Final results CSV saved
✓ Grad-CAM image saved
✓ Final model copied to Results folder

```

```

          CIFAKE CNN PERFORMANCE
Best Training Accuracy   : 90.83 %
Best Validation Accuracy : 88.53 %
Final Test Accuracy      : 90.07 %
Final Test Loss          : 0.2411

```

7.7 Misclassification Analysis

A total of 149 images were incorrectly classified.

These images were further examined to understand cases where the model had difficulty distinguishing between FAKE and REAL images.

The project also calculated prediction uncertainty using the distance of the REAL probability from the 0.5 classification threshold.

Images with probabilities close to 0.5 were treated as lower-confidence cases and can be useful for further analysis.

7.8 Explainability Results

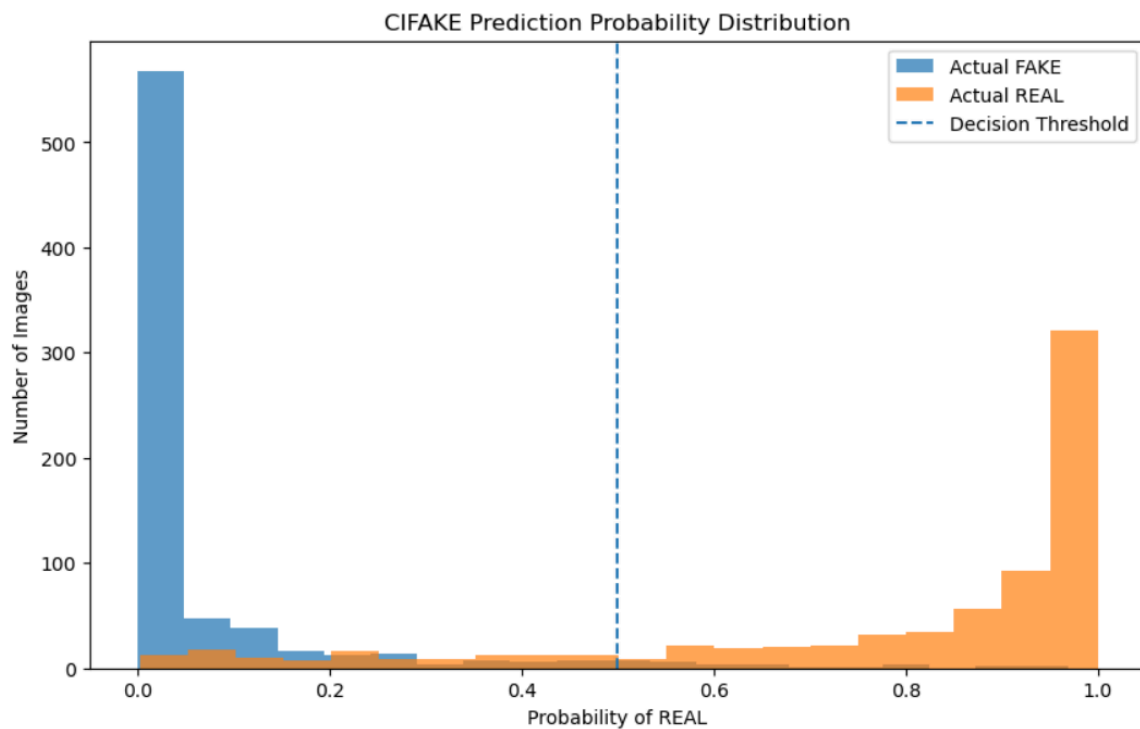
Grad-CAM was used to visualize important image regions associated with a prediction.

The output consists of:

1. Original image.
2. Grad-CAM heatmap.
3. Heatmap overlay on the original image.

This provides an additional layer of interpretation for the CNN prediction.

The Grad-CAM output does not prove that a particular region is the actual cause of an image being synthetic. Instead, it provides a visualization of regions associated with the model's learned prediction.



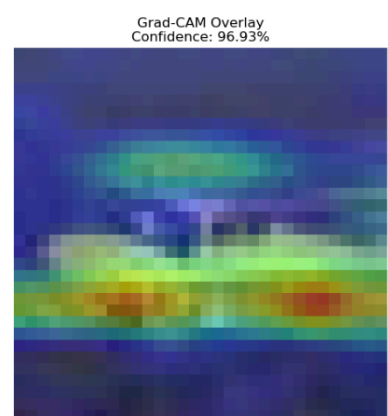
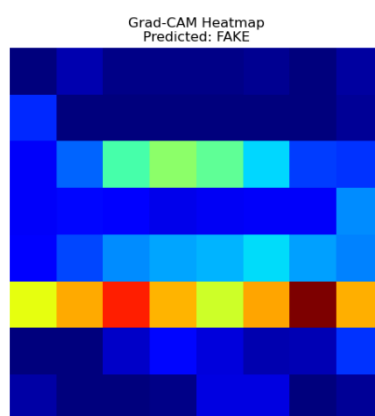
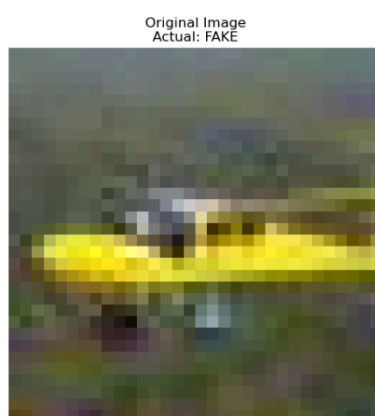
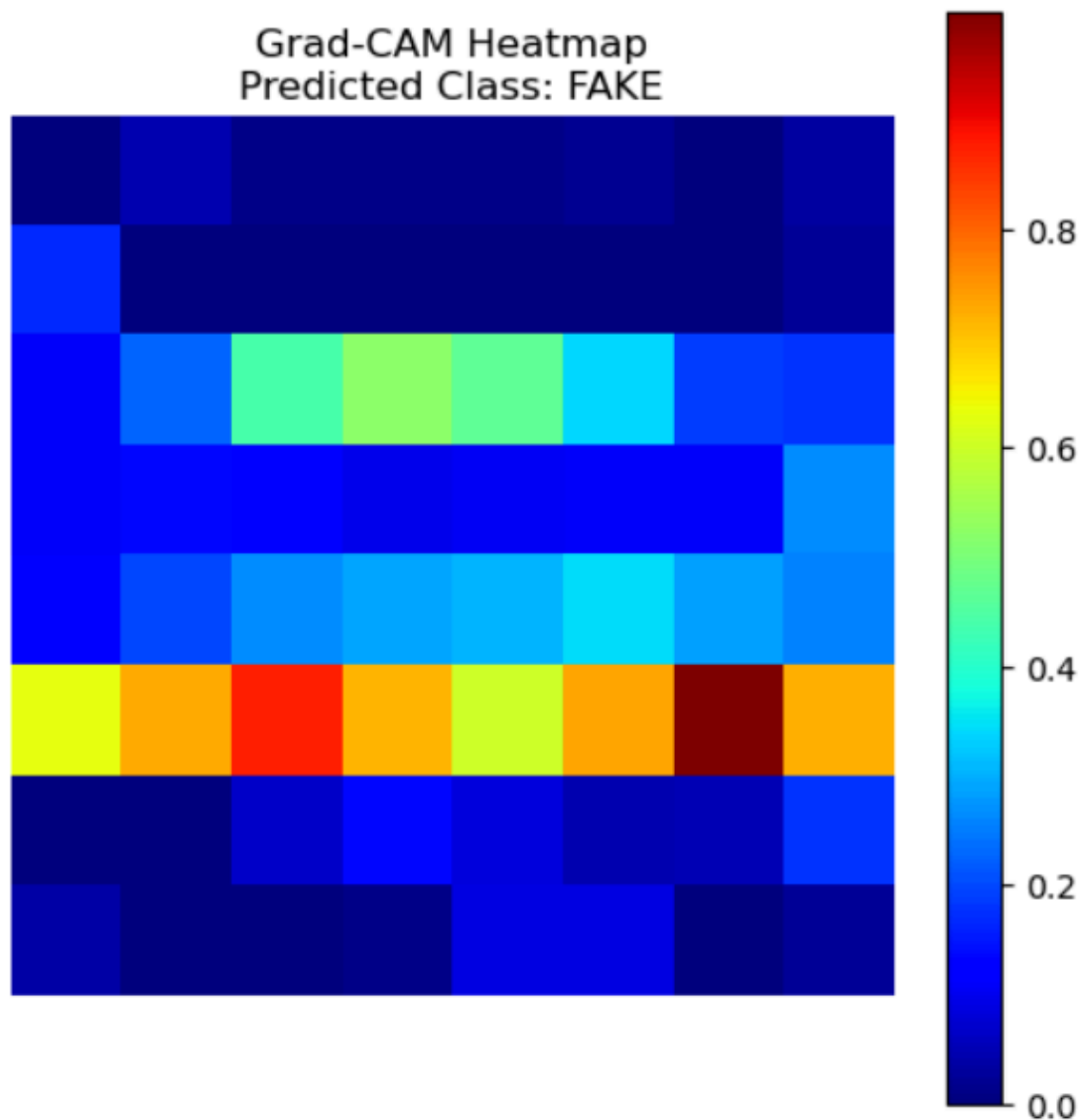
Classification Report ¶

```
: report = classification_report(  
    y_true,  
    y_pred,  
    target_names=class_names,  
    digits=4  
)  
  
print(report)
```

	precision	recall	f1-score	support
FAKE	0.8556	0.9640	0.9066	750
REAL	0.9588	0.8373	0.8940	750
accuracy			0.9007	1500
macro avg	0.9072	0.9007	0.9003	1500
weighted avg	0.9072	0.9007	0.9003	1500

Predicted: FAKE
Confidence: 71.20%
Actual: FAKE





Final Accuracy : 90.07%
Final F1-Score : 89.40%
Best Epoch : 4

CHAPTER 8: CHALLENGES AND LEARNING OUTCOMES

8.1 Challenges Faced

During the development of the project, several practical challenges were encountered.

Dataset File Handling

One challenge involved handling image file paths and ensuring that only valid image files were processed. Some file-list references became outdated during notebook execution, so the image lists had to be refreshed from the actual dataset folders.

```
Number of FAKE images: 50000
Number of REAL images: 50000
Total images: 100000
```

Image Validation

It was necessary to check image files for corruption before using them for training. This helped prevent invalid files from causing problems during preprocessing.



CNN Training

Selecting an appropriate model architecture, learning rate and training strategy required experimentation. Early stopping and checkpointing were used to make the training process more reliable.

```

Epoch 4: val_accuracy improved from 0.79067 to 0.88533, saving model to cifake_best_model.keras
110/110 ————— 7s 63ms/step - accuracy: 0.8839 - loss: 0.2799 - val_accuracy: 0.8853 - val_loss: 0.2650
Epoch 5/20
110/110 ————— 0s 58ms/step - accuracy: 0.8828 - loss: 0.2699
Epoch 5: val_accuracy did not improve from 0.88533
110/110 ————— 7s 64ms/step - accuracy: 0.8829 - loss: 0.2699 - val_accuracy: 0.7353 - val_loss: 0.6901
Epoch 6/20
109/110 ————— 0s 58ms/step - accuracy: 0.8959 - loss: 0.2542
Epoch 6: val_accuracy did not improve from 0.88533
110/110 ————— 7s 63ms/step - accuracy: 0.8959 - loss: 0.2542 - val_accuracy: 0.8520 - val_loss: 0.3572
Epoch 7/20
110/110 ————— 0s 58ms/step - accuracy: 0.8976 - loss: 0.2453
Epoch 7: val_accuracy did not improve from 0.88533
110/110 ————— 7s 64ms/step - accuracy: 0.8976 - loss: 0.2453 - val_accuracy: 0.8133 - val_loss: 0.4849
Epoch 8/20
109/110 ————— 0s 58ms/step - accuracy: 0.9038 - loss: 0.2293
Epoch 8: val_accuracy did not improve from 0.88533
110/110 ————— 7s 64ms/step - accuracy: 0.9039 - loss: 0.2292 - val_accuracy: 0.8233 - val_loss: 0.4848
Epoch 9/20
110/110 ————— 0s 59ms/step - accuracy: 0.9051 - loss: 0.2248
Epoch 9: val_accuracy did not improve from 0.88533
110/110 ————— 7s 65ms/step - accuracy: 0.9051 - loss: 0.2248 - val_accuracy: 0.8700 - val_loss: 0.3607
Epoch 9: early stopping
Restoring model weights from the end of the best epoch: 4.

```

```

print("Training completed!")

print("\nNumber of epochs actually completed:")
print(len(history.history["loss"]))

Training completed!

```

Grad-CAM Implementation

The most technically challenging part was implementing Grad-CAM with the trained Keras model.

The original Sequential model produced graph and input/output connectivity issues when intermediate convolutional outputs were accessed for Grad-CAM. To solve this, a separate functional model was constructed using the same trained layer configurations and weights.

This allowed the final convolutional feature maps to be connected properly to the model output for gradient calculation.

Creating the Grad-CAM model

```
grad_model = tf.keras.Model(  
    inputs=gradcam_model.input,  
    outputs=[  
        last_conv_output,  
        gradcam_model.output  
    ]  
)  
  
print("Grad-CAM model created successfully.")  
print("Input shape :", grad_model.input_shape)  
print("Output count:", len(grad_model.outputs))
```

```
Grad-CAM model created successfully.  
Input shape : (None, 32, 32, 3)  
Output count: 2
```

Output File Handling

Another practical issue occurred while saving result images because of the deeply nested Windows project path. Although the directory was present and writable, saving through Matplotlib produced a file-path error.

A shorter results directory was therefore used to reliably store the generated output files.

8.2 Learning Outcomes

The project provided practical experience in several areas:

- Python programming.
- Image dataset management.
- Data preprocessing.
- CNN architecture design.
- TensorFlow/Keras.
- Model training and validation.
- Binary classification.
- Performance evaluation.
- Confusion matrix interpretation.
- Prediction confidence analysis.
- Explainable AI.
- Grad-CAM implementation.
- Data visualization.

- Model saving and deployment preparation.
- Debugging machine learning code.

The project also provided experience in understanding the complete lifecycle of a deep learning project rather than focusing only on model training.

CHAPTER 9: FUTURE ENHANCEMENTS

The current project can be extended in several ways.

9.1 Higher-Resolution Images

The current model uses 32×32 pixel images. Future implementations could experiment with larger image resolutions to preserve more visual information.

9.2 Transfer Learning

Pre-trained architectures such as ResNet, EfficientNet or MobileNet could be evaluated for comparison with the custom CNN.

Transfer learning may allow the system to benefit from features learned from larger image datasets.

9.3 Larger and More Diverse Datasets

The model could be trained and evaluated using additional datasets containing AI-generated images from different generation techniques.

This would provide a broader test of how the model performs on images outside the original dataset.

9.4 Generator-Specific Classification

Instead of only predicting FAKE or REAL, a future system could attempt to identify the source or category of synthetic images, such as images generated by different types of generative models.

9.5 Advanced Explainability

Additional XAI techniques could be compared with Grad-CAM. Methods such as Integrated Gradients, LIME or SHAP could provide alternative explanations.

9.6 Web Application

The trained model could be integrated into a simple web application using technologies such as Streamlit.

A user could upload an image and receive:

- Predicted class.
- Confidence/probability.
- Grad-CAM visualization.
- Explanation of the model output.

9.7 Robustness Testing

Future work could evaluate the model under image compression, resizing, cropping, brightness changes and other modifications.

This would help determine how stable the predictions are when images differ from the training data.

9.8 Continuous Dataset Updates

As AI-generated images continue to improve, new examples could be added to the training dataset. Periodic retraining could help the system adapt to new types of synthetic content.

CHAPTER 10: CONCLUSION

The project “**CIFAKE Image Classification and Explainable Identification of AI-Generated Synthetic Images**” presented an end-to-end deep learning approach for distinguishing between FAKE and REAL images.

The project began with dataset validation and preparation, followed by image preprocessing, augmentation and stratified dataset splitting. A CNN-based binary classifier was then developed using TensorFlow and Keras.

The trained model was evaluated on 1,500 test images and achieved an accuracy of 90.07%, with a precision of 95.88%, recall of 83.73%, and F1-score of 89.40%. The confusion matrix showed 1,351 correct classifications and 149 incorrect classifications.

The project also incorporated Grad-CAM to provide a visual explanation of CNN predictions. This was an important part of the project because it allowed the classification results to be examined beyond a simple FAKE or REAL output.

One of the major technical challenges was connecting the trained CNN's intermediate convolutional features to a Grad-CAM model in the Keras environment. Resolving this issue provided practical experience with neural-network architecture, model graphs, intermediate feature maps and gradient-based explanations.

Overall, the project demonstrates how deep learning, image classification and explainable AI can be combined into a single practical workflow for analysing AI-generated and real images. It also provided valuable hands-on experience with dataset preparation, CNN development, model evaluation, visualization, debugging and XAI.

The current implementation provides a foundation that can be further developed using larger datasets, higher-resolution images, transfer learning, advanced explainability techniques and a user-friendly web application.